Open-Source Software for Quantum Many-Body Computation

State-of-the-Art Review

"survey review draft"

Generated 2026-06-20 from a 22-package knowledge base.

Scope

This report assesses the landscape of *popular open-source software* for computing ground-state and finite-temperature properties of quantum many-body lattice models. It is written for a researcher or team choosing a code, or onboarding to the field, and is organized *by technical approach* (method family): one code is rarely "best" — the right tool is set by the model, dimensionality, and target observable. It covers 22 actively-maintained packages with a published release paper; it deliberately excludes density-functional / electronic-structure suites, closed-source codes, and one-off group scripts. State of the art and trade-offs are given *inside each approach*, not as global sections.

1. What and Why

The quantum many-body problem is the task of computing properties of interacting quantum systems — spins, bosons, or fermions on a lattice — whose Hilbert space grows *exponentially* with system size. A modest 50-site spin-½ model already has $2^{50} \approx 10^{15}$ basis states, so brute-force storage of the wavefunction is impossible beyond 20 sites. Every practical method is therefore a *controlled compression* of that exponential object: a tensor-network ansatz, a stochastic sample, a variational parametrization, or a self-consistent local approximation. Many-body computation software packages each implement one such compression, plus the lattice/Hamiltonian bookkeeping, symmetry handling, and convergence diagnostics needed to use it reliably.

This matters now because the field has shifted from private, per-group Fortran/C++ codes to a shared, documented, *citable* open-source ecosystem. Reproducibility pressure, the rise of publication venues for research software (e.g. SciPost Physics Codebases), and a wave of high-productivity numerical languages have lowered the barrier to entry: a graduate student today runs DMRG [1] or neural-network states [2] in a notebook rather than porting a legacy code. A recent survey of the DMRG-codes landscape documents both the breadth of this ecosystem and its growing pains [3].

How it differs from the prior approach: instead of a single monolithic solver, the modern stack is a set of *method-specialized* libraries — DMRG/tensor networks for 1D and gapped 2D systems, quantum Monte Carlo for sign-problem-free regimes, exact diagonalization for small frustrated clusters, variational/neural states for 2D frustrated and continuous-time dynamics, and dynamical mean-field theory for correlated electronic structure. The clear recent trend is toward *Julia + automatic differentiation* and *Python + JAX*, which wrap or replace older compiled cores while keeping their performance.

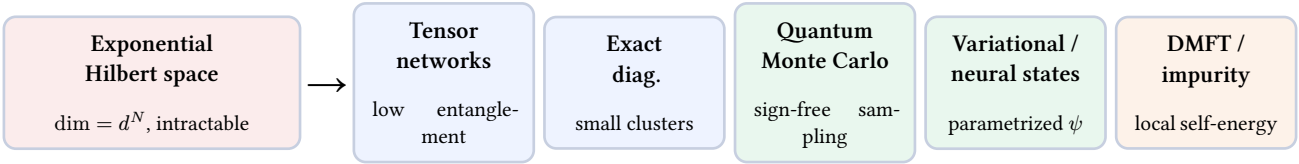

Figure 1: The shared problem (left) and the five method families that compress it (right). Each family is implemented by one or more of the surveyed packages; none dominates across all regimes.

2. Technical Approaches

The ecosystem clusters into five method families. A clear generational shift runs through all of them: compiled C++/Fortran cores from 2011–2017 have been progressively wrapped or replaced by Julia and Python+JAX packages from 2020 onward, increasingly published through SciPost Physics Codebases.

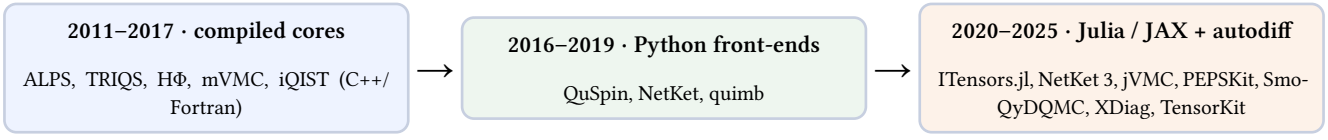

Figure 2: Field landscape: from monolithic compiled solvers to composable autodiff-native libraries.

2.1. Tensor networks (DMRG / MPS and PEPS)

What it is. Tensor-network methods represent the wavefunction as a contraction of small tensors whose bond dimension χ caps the captured entanglement. In one dimension the matrix-product-state (MPS) ansatz with the density-matrix renormal-

ization group (DMRG) is essentially exact for gapped systems; in two dimensions the projected-entangled-pair-state (PEPS) generalization trades the clean 1D algorithms for harder contraction and optimization.

State of the art. For 1D and quasi-2D ground states and dynamics, ITensor (C++ and Julia) is the general-purpose workhorse [1], with TeNPy the leading Python alternative [4] and quimb adding large-scale, contraction-path-optimized tensor networks [5]. block2 pushes high-performance DMRG with a strong quantum-chemistry, finite-T, and dynamics focus [6]. The Julia stack is now unified by TensorKit.jl, a symmetric-tensor backend supporting Abelian, non-Abelian, and anyonic symmetries [7]. In 2D, the current reference is automatic-differentiation–based infinite-PEPS optimization (CTMRG environments with reverse-mode AD), as implemented in the PEPSKit.jl line of work [8]. A 2025 survey maps the full DMRG-codes ecosystem and its maturity spread [3].

Strengths
- Near-exact, variational (true energy upper bound) for 1D/gapped systems, with controlled error via χ [1].
- No sign problem — works for frustrated and fermionic models inaccessible to QMC [4].
- Mature, well-documented 1D tooling with symmetry support across languages [7].

Limitations
- Cost grows steeply with entanglement ($\sim \chi^3$); 2D and critical systems need large χ [3].
- PEPS optimization (gauge fixing, CTMRG convergence, AD stability) is far less turnkey than 1D DMRG [8].
- Real-time dynamics is limited by entanglement growth [4].

2.2. Exact diagonalization

What it is. Exact diagonalization (ED) builds the Hamiltonian matrix in a symmetry-reduced basis and extracts low-lying eigenpairs (Lanczos) or the full spectrum. It is the unbiased reference method — no ansatz, no sign problem — but limited to small clusters by the exponential basis size.

State of the art. QuSpin is the most widely used package for ED and quantum dynamics of spin, boson, and fermion lattices, with a Python interface and extensive symmetry support [9]. XDiag (C++ core with a Julia wrapper) pushes the size frontier through symmetry-adapted bases and sublattice coding [10]. HΦ adds full diagonalization and thermal-pure-quantum (TPQ) sampling for finite-temperature properties of Hubbard, Heisenberg, and Kondo models [11]. QuTiP, though built for open-system dynamics, is a common choice for small-system ED and Hamiltonian construction [12].

Strengths
- Unbiased and exact — the gold-standard benchmark for other methods [9].
- No sign problem; handles arbitrary frustration, disorder, and long-range terms [10].
- Gives the full spectrum / dynamical correlators and finite-T via TPQ [11].

Limitations
- Exponential memory/time — practically capped near 40–50 spin-½ sites even with symmetries [10].
- Finite-size effects dominate; extrapolation to the thermodynamic limit is indirect [9].

2.3. Quantum Monte Carlo

What it is. Quantum Monte Carlo (QMC) maps the quantum partition function to a classical statistical sum and samples it stochastically — worldline / stochastic-series-expansion for spins and bosons, or determinant / auxiliary-field for fermions. It scales polynomially and reaches large sizes, but is restricted to models without a fermion (or frustration) sign problem.

State of the art. ALPS is the historical meta-framework bundling loop/SSE/worm QMC alongside ED and DMRG [13]. ALF provides finite-temperature and projective auxiliary-field (determinant) QMC for interacting fermions [14]. SmoQyDQMC.jl is a modern Julia determinant-QMC code for Hubbard and electron–phonon models, including anharmonic and acoustic phonons via hybrid Monte Carlo [15]. DSQSS implements worldline / directed-loop QMC for spin and Bose-Hubbard systems at finite temperature [16].

Strengths
- Polynomial scaling — reaches thousands of sites, enabling thermodynamic-limit extrapolation [14].
- Numerically exact (statistical errors only) for sign-free models [16].
- Natural finite-temperature access [15].

Limitations
- Fermion / frustration sign problem makes doped and frustrated models exponentially hard [14].
- Real-frequency dynamics needs ill-posed analytic continuation [13].
- Autocorrelation and equilibration can be costly near critical points [16].

2.4. Variational Monte Carlo and neural quantum states

What it is. Variational Monte Carlo (VMC) optimizes a parametrized trial wavefunction by sampling. Neural quantum states (NQS) replace the traditional Jastrow/pair-product form with a neural network, giving a flexible ansatz whose parameters are trained by stochastic reconfiguration or gradient descent. This family targets exactly the frustrated 2D and fermionic regimes where QMC fails and tensor networks strain.

State of the art. NetKet (Python, JAX) is the dominant NQS toolbox for ground states and dynamics, with composable network architectures and autodiff [2]. jVMC offers a GPU-accelerated, AD-based VMC implementation in the same JAX ecosystem [17]. mVMC is the established many-variable VMC code for interacting fermions, optimizing 10^4+ parameters via stochastic reconfiguration for Hubbard, Heisenberg, and Kondo models [18].

Strengths

- Flexible ansatz reaches 2D frustrated and fermionic systems beyond QMC and PEPS [2].
- Variational upper bound; GPU/autodiff-native and scalable [17].
- Mature fermionic VMC with many-parameter optimization [18].

Limitations

- Optimization is non-convex; convergence and reliability diagnostics lag tensor networks [2].
- Expressivity-vs-cost trade-off is poorly characterized; results can be hard to certify [17].

2.5. Dynamical mean-field theory and impurity solvers

What it is. Dynamical mean-field theory (DMFT) maps a correlated lattice model onto a self-consistently determined quantum impurity embedded in a bath, capturing local quantum fluctuations exactly. The computational core is the impurity solver — most often continuous-time quantum Monte Carlo (CT-HYB). DMFT is the workhorse for correlated electronic structure and realistic materials.

State of the art. TRIQS is the dominant toolbox for interacting quantum systems and DMFT, with a C++/Python interface and DFT interfaces [19]. w2dynamics provides a multi-orbital CT-HYB solver with a full DMFT loop [20], and iQIST offers CT-HYB and Hirsch–Fye impurity solvers [21]. DCore integrates these into a turnkey DMFT package with DFT front-ends (via Wannier90) [22].

Strengths

- Handles multi-orbital, realistic materials by coupling to DFT [19].
- Captures local correlations (Mott transition, quasiparticles) at modest cost [20].
- Mature, modular ecosystem with integrated workflows [22].

Limitations

- Neglects non-local correlations unless extended (cluster/diagrammatic) at much higher cost [19].
- Impurity-solver QMC inherits sign problems and analytic-continuation issues [21].

2.6. At-a-glance comparison

Approach	Representative codes	Reach	Key limitation	Best-fit use
Tensor networks	ITensor [1], TeNPy [4], PEPSKit [8]	1D exact; 2D harder	Entanglement / χ cost; 2D not turnkey [3]	Gapped 1D & quasi-2D, frustrated spins
Exact diagonalization	QuSpin [9], XDiag [10], HΦ [11]	40–50 sites	Exponential size limit [10]	Small frustrated clusters; benchmarks
Quantum Monte Carlo	ALF [14], SmoQyDQMC [15], DSQSS [16]	1000s of sites	Sign problem [14]	Sign-free spin/boson & half-filled fermions, finite T
VMC / neural states	NetKet [2], jVMC [17], mVMC [18]	Large 2D	Non-convex; hard to certify [2]	Frustrated 2D & fermionic where QMC fails
DMFT / impurity	TRIQS [19], w2dynamics [20], DCore [22]	Bulk + materials	Local approx.; solver sign problem [21]	Correlated electronic structure, materials

3. Open Problems

No.	Problem	Why it matters	Who could solve it	Urgency
1	Fermion / frustration sign problem	Doped Hubbard and frustrated magnets — the central open questions in correlated matter — stay exponentially hard for QMC, forcing reliance on less-controlled methods [14].	QMC + tensor-network / NQS developers; algorithmic theory	Critical
2	Robust, turnkey 2D tensor networks	PEPS optimization (gauge fixing, CTMRG convergence, AD stability) remains research-grade, blocking routine 2D ground-state studies [3], [8].	PEPSKit / TensorKit groups [7]	Critical
3	Certifying neural-quantum-state results	NQS reach regimes others cannot, but lack the convergence guarantees and error bars that make tensor-network and QMC results trustworthy [2], [17].	NetKet / jVMC + ML-theory community	High
4	Interoperability and common formats	Codes use incompatible Hamiltonian/lattice/symmetry representations, so cross-method validation — the backbone of trust — is manual and error-prone [3].	Cross-package consortium; standards effort	High
5	GPU / HPC modernization of legacy cores	JAX/Julia codes get GPU acceleration cheaply, but mature C++/Fortran solvers (ALPS, TRIQS, mVMC) need heavy porting to keep pace [13], [18].	Core maintainers + HPC engineers	Medium
6	Sustainability and citation hygiene	Many packages are single-group projects with uneven maintenance; widely-used tools (MPSKit.jl, peps-torch, pomerol) lack a standalone release paper, complicating provenance [3].	Funders, journals (SciPost Codebases), communities	Medium

Bottom line

There is no single best many-body code — the field is a portfolio of method-specialized libraries, and the right choice is dictated by dimensionality, sign structure, and target observable: tensor networks (ITensor/TeNPy) for 1D and gapped systems, ED (QuSpin/XDiag) for small frustrated clusters and benchmarks, QMC (ALF/SmoQyDQMC) for sign-free models at scale, neural/variational states (NetKet/jVMC) for frustrated 2D where QMC fails, and DMFT (TRIQS/DCore) for correlated materials. The strongest current momentum is in the Julia+autodiff and Python+JAX ecosystems [2], [7]; the highest-value open targets are the sign problem and turnkey 2D tensor networks [8], [14].

References

[1] M. Fishman, S. White, and E. Stoudenmire, "The ITensor Software Library for Tensor Network Calculations," *ArXiv*, 2020, doi: 10.21468/SciPostPhysCodeb.4.

[2] F. Vicentini *et al.*, "NetKet 3: Machine Learning Toolbox for Many-Body Quantum Systems," *ArXiv*, 2021, doi: 10.21468/SciPostPhysCodeb.7.

[3] P. Sehlstedt, J. Brandejs, P. Bientinesi, and L. Karlsson, "The Software Landscape for the Density Matrix Renormalization Group," *Comput. Phys. Commun.*, 2025, doi: 10.1016/j.cpc.2026.110136.

[4] J. Hauschild *et al.*, "Tensor network Python (TeNPy) version 1," *SciPost Physics Codebases*, 2024, doi: 10.21468/SciPostPhysCodeb.41.

[5] J. Gray, "quimb: A python package for quantum information and many-body calculations," *J. Open Source Softw.*, 2018, doi: 10.21105/JOSS.00819.

[6] H. Zhai *et al.*, "Block2: A comprehensive open source framework to develop and apply state-of-the-art DMRG algorithms in electronic structure and beyond.," *The Journal of chemical physics*, 2023, doi: 10.1063/5.0180424.

[7] L. Devos and J. Haegeman, "TensorKit.jl: A Julia package for large-scale tensor computations, with a hint of category theory," *ArXiv*, 2025, doi: 10.48550/arXiv.2508.10076.

[8] J. Naumann, E. L. Weerda, M. Rizzi, J. Eisert, and P. Schmoll, "An introduction to infinite projected entangled-pair state methods for variational ground state simulations using automatic differentiation," *SciPost Physics Lecture Notes*, 2023, doi: 10.21468/SciPostPhysLectNotes.86.

[9] P. Weinberg and M. Bukov, "QuSpin: a Python Package for Dynamics and Exact Diagonalisation of Quantum Many Body Systems part I: spin chains," *arXiv: Computational Physics*, 2016, doi: 10.21468/SciPostPhys.2.1.003.

[10] A. Wietek *et al.*, "XDiag: Exact diagonalization for quantum many-body systems," *SciPost Physics Codebases*, 2025, doi: 10.21468/SciPostPhysCodeb.70.

[11] M. Kawamura, K. Yoshimi, T. Misawa, Y. Yamaji, S. Todo, and N. Kawashima, "Quantum lattice model solver HΦ," *Comput. Phys. Commun.*, 2017, doi: 10.1016/j.cpc.2017.04.006.

[12] J. Johansson, P. Nation, and F. Nori, "QuTiP: An open-source Python framework for the dynamics of open quantum systems," *Comput. Phys. Commun.*, 2011, doi: 10.1016/j.cpc.2012.02.021.

[13] B. Bauer *et al.*, "The ALPS project release 2.0: open source software for strongly correlated systems," *Journal of Statistical Mechanics: Theory and Experiment*, 2011, doi: 10.1088/1742-5468/2011/05/P05001.

[14] M. Bercx, F. Goth, J. S. Hofmann, and F. Assaad, "The ALF (Algorithms for Lattice Fermions) project release 1.0. Documentation for the auxiliary field quantum Monte Carlo code," *arXiv: Strongly Correlated Electrons*, 2017, doi: 10.21468/SciPostPhys.3.2.013.

[15] B. Cohen-Stead *et al.*, "SmoQyDQMC.jl: A flexible implementation of determinant quantum Monte Carlo for Hubbard and electron-phonon interactions," *SciPost Physics Codebases*, 2023, doi: 10.21468/SciPostPhysCodeb.29.

[16] Y. Motoyama, K. Yoshimi, A. Masaki-Kato, T. Kato, and N. Kawashima, "DSQSS: Discrete Space Quantum Systems Solver," *Comput. Phys. Commun.*, 2020, doi: 10.1016/J.CPC.2021.107944.

[17] M. Schmitt and M. Reh, "jVMC: Versatile and performant variational Monte Carlo leveraging automated differentiation and GPU acceleration," *SciPost Physics Codebases*, 2021, doi: 10.21468/SciPostPhysCodeb.2.

[18] T. Misawa *et al.*, "mVMC - Open-source software for many-variable variational Monte Carlo method," *Comput. Phys. Commun.*, 2017, doi: 10.1016/j.cpc.2018.08.014.

[19] O. Parcollet *et al.*, "TRIQS: A toolbox for research on interacting quantum systems," *Comput. Phys. Commun.*, 2015, doi: 10.1016/j.cpc.2015.04.023.

[20] M. Wallerberger *et al.*, "w2dynamics: Local one- and two-particle quantities from dynamical mean field theory," *Comput. Phys. Commun.*, 2018, doi: 10.1016/j.cpc.2018.09.007.

[21] L. Huang, Y. Wang, Z. Meng, L. Du, P. Werner, and X. Dai, "iQIST: An open source continuous-time quantum Monte Carlo impurity solver toolkit," *Comput. Phys. Commun.*, 2014, doi: 10.1016/j.cpc.2015.04.020.

[22] H. Shinaoka, J. Otsuki, M. Kawamura, N. Takemori, and K. Yoshimi, "DCore: Integrated DMFT software for correlated electrons," *SciPost Physics*, 2020, doi: 10.21468/SciPostPhys.10.5.117.